

CSE 314: Software Engineering Sessional

Load Testing with JMeter

Nafis Tahmid

Department of CSE, BUET

What is JMeter, and why should we bother?

First: what problem are we solving?

Your website works. You know this because you opened it once, and it did the thing.

First: what problem are we solving?

Your website works. You know this because you opened it once, and it did the thing.

Now consider:

- What happens when **50 people** open it at once?

First: what problem are we solving?

Your website works. You know this because you opened it once, and it did the thing.

Now consider:

- What happens when **50 people** open it at once?
- What about **500? 5,000?**

First: what problem are we solving?

Your website works. You know this because you opened it once, and it did the thing.

Now consider:

- What happens when **50 people** open it at once?
- What about **500? 5,000?**
- Where does it slow down? Where does it break?

First: what problem are we solving?

Your website works. You know this because you opened it once, and it did the thing.

Now consider:

- What happens when **50 people** open it at once?
- What about **500? 5,000?**
- Where does it slow down? Where does it break?
- How long can it sustain that load before something catches fire?

First: what problem are we solving?

Your website works. You know this because you opened it once, and it did the thing.

Now consider:

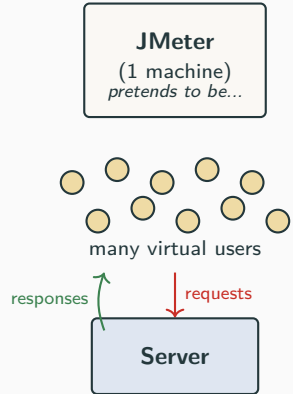
- What happens when **50 people** open it at once?
- What about **500? 5,000?**
- Where does it slow down? Where does it break?
- How long can it sustain that load before something catches fire?



These are questions of **performance**, not **correctness**. A perfectly correct application can still be perfectly unusable. Performance testing exists so we find that out on a Tuesday afternoon, and not at 3 AM the night before launch.

So, what is JMeter?

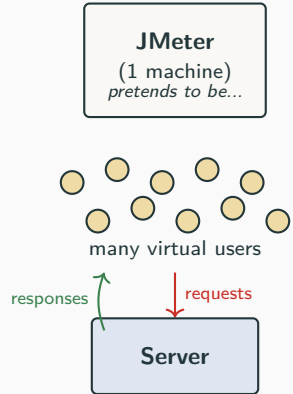
Apache JMeter is a tool that pretends to be a large number of users, hits your server with their pretend traffic, and reports back on what happened.



So, what is JMeter?

Apache JMeter is a tool that pretends to be a large number of users, hits your server with their pretend traffic, and reports back on what happened.

Key facts you will need to know:

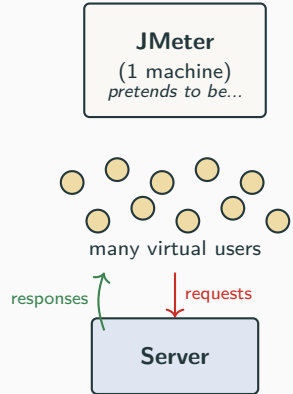


So, what is JMeter?

Apache JMeter is a tool that pretends to be a large number of users, hits your server with their pretend traffic, and reports back on what happened.

Key facts you will need to know:

- **Performance test application** — built for load, stress and endurance testing.

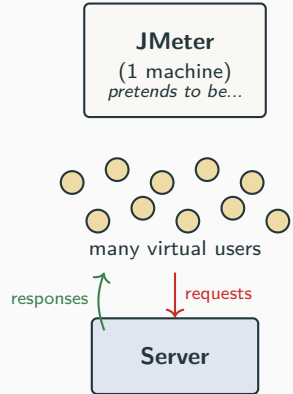


So, what is JMeter?

Apache JMeter is a tool that pretends to be a large number of users, hits your server with their pretend traffic, and reports back on what happened.

Key facts you will need to know:

- **Performance test application** — built for load, stress and endurance testing.
- **Written in Java.** You will need a JDK. This is non-negotiable.

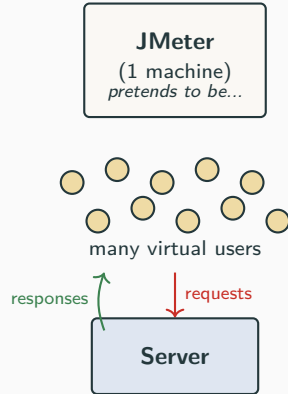


So, what is JMeter?

Apache JMeter is a tool that pretends to be a large number of users, hits your server with their pretend traffic, and reports back on what happened.

Key facts you will need to know:

- **Performance test application** — built for load, stress and endurance testing.
- **Written in Java.** You will need a JDK. This is non-negotiable.
- **Free and open source.** Apache License 2.0.

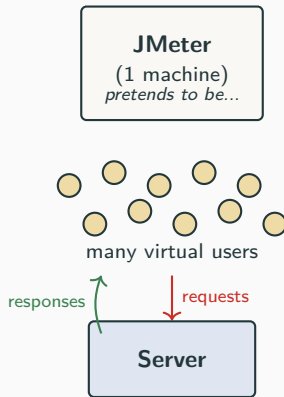


So, what is JMeter?

Apache JMeter is a tool that pretends to be a large number of users, hits your server with their pretend traffic, and reports back on what happened.

Key facts you will need to know:

- **Performance test application** — built for load, stress and endurance testing.
- **Written in Java.** You will need a JDK. This is non-negotiable.
- **Free and open source.** Apache License 2.0.
- Supports HTTP(S), FTP, JDBC, SOAP/REST, JMS, TCP, and several other things you may never touch.

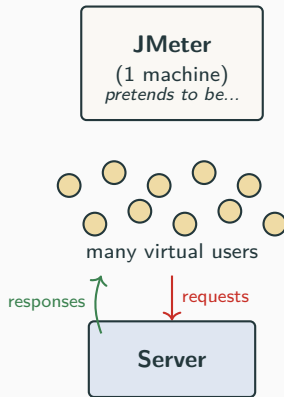


So, what is JMeter?

Apache JMeter is a tool that pretends to be a large number of users, hits your server with their pretend traffic, and reports back on what happened.

Key facts you will need to know:

- **Performance test application** — built for load, stress and endurance testing.
- **Written in Java.** You will need a JDK. This is non-negotiable.
- **Free and open source.** Apache License 2.0.
- Supports HTTP(S), FTP, JDBC, SOAP/REST, JMS, TCP, and several other things you may never touch.
- Has a **GUI** for building tests, a **CLI** for running them, and a **recorder**.



What JMeter is *not*

A gentle disambiguation, because these confusions are widespread.

- **Not a browser.** JMeter speaks the protocol (HTTP), not the rendering. It will not execute your JavaScript, it will not paint your CSS, and it will absolutely not click a button. It sends requests and reads responses.

What JMeter is *not*

A gentle disambiguation, because these confusions are widespread.

- **Not a browser.** JMeter speaks the protocol (HTTP), not the rendering. It will not execute your JavaScript, it will not paint your CSS, and it will absolutely not click a button. It sends requests and reads responses.
- **Not a functional testing tool** — although you can add assertions. Selenium, Cypress and their friends live in a different neighbourhood.

What JMeter is *not*

A gentle disambiguation, because these confusions are widespread.

- **Not a browser.** JMeter speaks the protocol (HTTP), not the rendering. It will not execute your JavaScript, it will not paint your CSS, and it will absolutely not click a button. It sends requests and reads responses.
- **Not a functional testing tool** — although you can add assertions. Selenium, Cypress and their friends live in a different neighbourhood.
- **Not magic.** If you point 10,000 threads at a machine you don't own, you are performing a DDoS attack with extra steps. Get permission first.

What JMeter is *not*

A gentle disambiguation, because these confusions are widespread.

- **Not a browser.** JMeter speaks the protocol (HTTP), not the rendering. It will not execute your JavaScript, it will not paint your CSS, and it will absolutely not click a button. It sends requests and reads responses.
- **Not a functional testing tool** — although you can add assertions. Selenium, Cypress and their friends live in a different neighbourhood.
- **Not magic.** If you point 10,000 threads at a machine you don't own, you are performing a DDoS attack with extra steps. Get permission first.



“But it *looks* like a browser hit” — yes, from the server's perspective, an HTTP request is an HTTP request. That is what makes JMeter useful, and why you must get written permission before pointing it at anything you don't own.

Installing the thing

Installation, in four unremarkable steps

1. Check that Java is installed.

```
java --version
```

If this prints a version, you're set. If it prints `command not found`, you have a small errand to run first (install a JDK, preferably 8 or newer).

Installation, in four unremarkable steps

1. Check that Java is installed.

```
java --version
```

If this prints a version, you're set. If it prints `command not found`, you have a small errand to run first (install a JDK, preferably 8 or newer).

2. **Download JMeter** from <https://jmeter.apache.org/>. Take the binary release.

Installation, in four unremarkable steps

1. Check that Java is installed.

```
java --version
```

If this prints a version, you're set. If it prints `command not found`, you have a small errand to run first (install a JDK, preferably 8 or newer).

2. **Download JMeter** from <https://jmeter.apache.org/>. Take the binary release.
3. **Unzip** the archive and put the folder somewhere you'll remember. There is no installer. There is nothing to install.

Installation, in four unremarkable steps

1. Check that Java is installed.

```
java --version
```

If this prints a version, you're set. If it prints `command not found`, you have a small errand to run first (install a JDK, preferably 8 or newer).

2. **Download JMeter** from <https://jmeter.apache.org/>. Take the binary release.
3. **Unzip** the archive and put the folder somewhere you'll remember. There is no installer. There is nothing to install.
4. **Start it.**
 - Windows: double-click `jmeter/bin/jmeter.bat`
 - macOS / Linux: `./jmeter/bin/jmeter.sh` from a terminal

Installation, in four unremarkable steps

1. Check that Java is installed.

```
java --version
```

If this prints a version, you're set. If it prints `command not found`, you have a small errand to run first (install a JDK, preferably 8 or newer).

2. **Download JMeter** from <https://jmeter.apache.org/>. Take the binary release.

3. **Unzip** the archive and put the folder somewhere you'll remember. There is no installer. There is nothing to install.

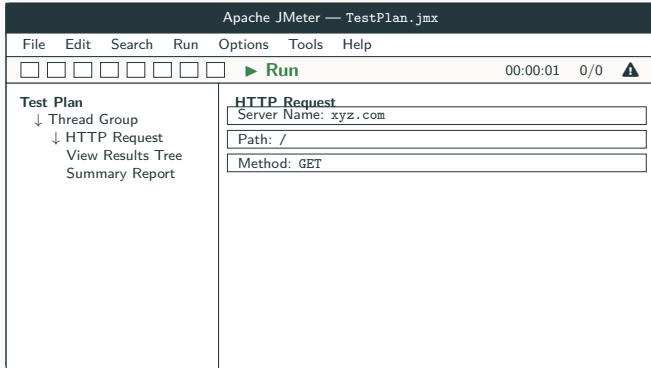
4. **Start it.**

- Windows: double-click `jmeter/bin/jmeter.bat`
- macOS / Linux: `./jmeter/bin/jmeter.sh` from a terminal

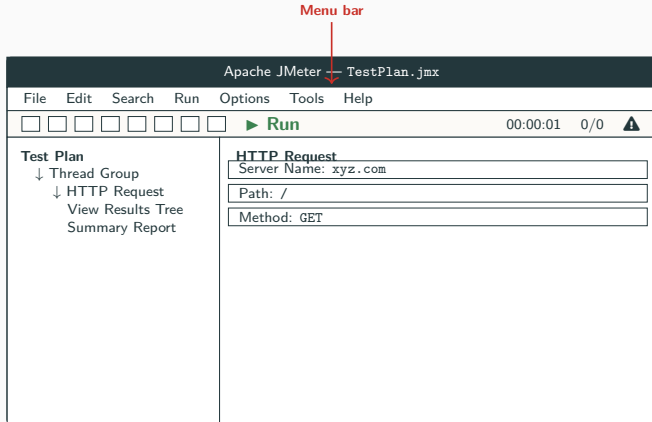


It will complain in the terminal about not being for load testing in GUI mode. It is telling the truth. We'll come back to that when we get to the CLI section.

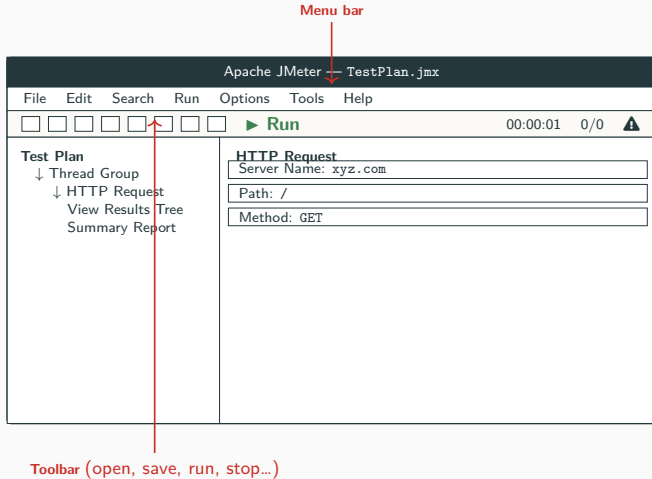
A tour of the GUI



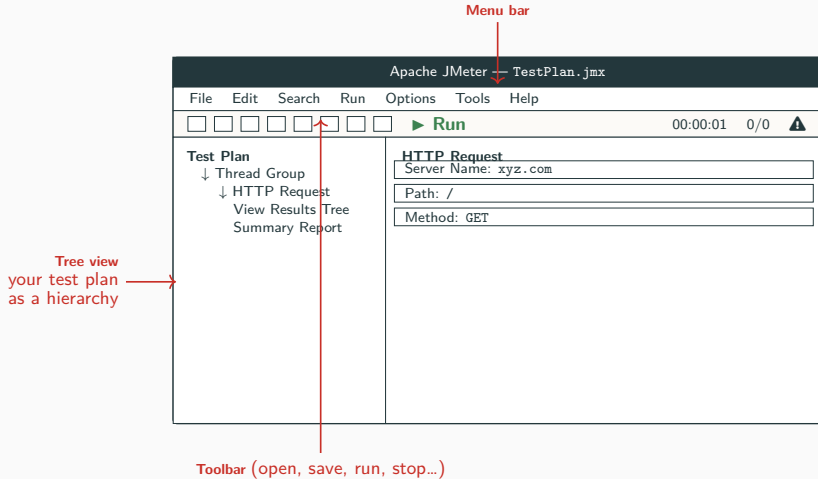
A tour of the GUI



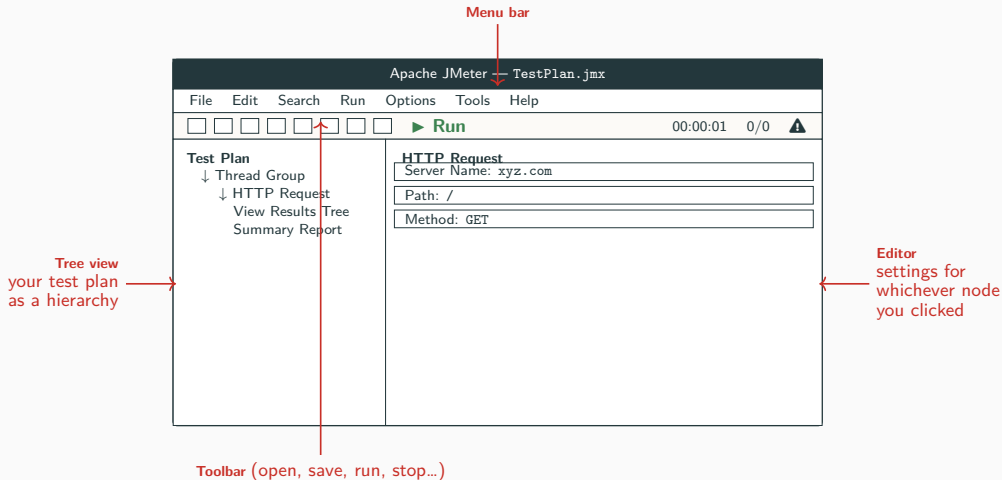
A tour of the GUI



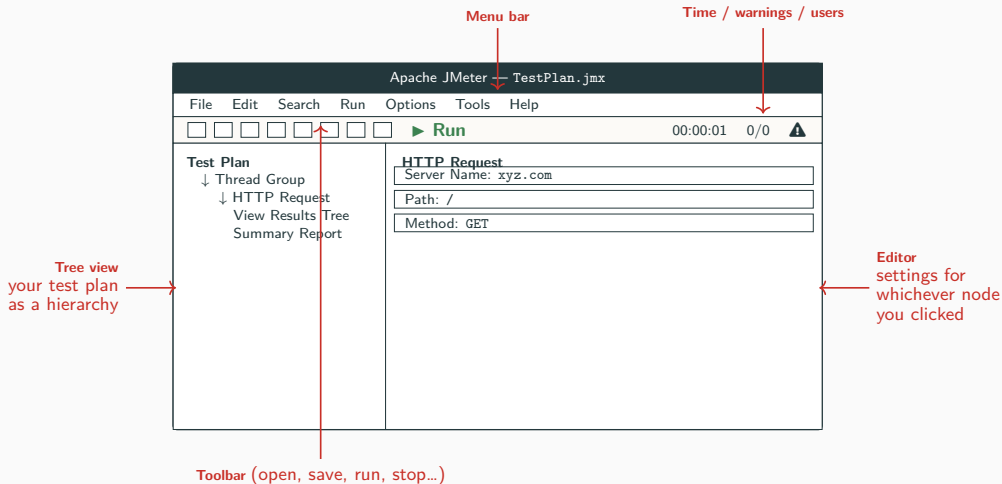
A tour of the GUI



A tour of the GUI



A tour of the GUI



The test plan: what it is, what goes in it

What is a test plan?

A **test plan** is a description of what JMeter should do when you press Run.

It answers three questions:

What is a test plan?

A **test plan** is a description of what JMeter should do when you press Run.

It answers three questions:

- **Who** is going to hit the server? (how many virtual users, arriving at what pace)

What is a test plan?

A **test plan** is a description of what JMeter should do when you press Run.

It answers three questions:

- **Who** is going to hit the server? (how many virtual users, arriving at what pace)
- **What** are they going to do? (which requests, in which order, with what data)

What is a test plan?

A **test plan** is a description of what JMeter should do when you press Run.

It answers three questions:

- **Who** is going to hit the server? (how many virtual users, arriving at what pace)
- **What** are they going to do? (which requests, in which order, with what data)
- **How** do we know what happened? (which results to collect, which conditions to check)

What is a test plan?

A **test plan** is a description of what JMeter should do when you press Run.

It answers three questions:

- **Who** is going to hit the server? (how many virtual users, arriving at what pace)
- **What** are they going to do? (which requests, in which order, with what data)
- **How** do we know what happened? (which results to collect, which conditions to check)

It is stored in a `.jmx` file (which is really just XML). You can commit it to Git, diff it, and be quietly disappointed with the diffs. It's XML.

What is a test plan?

A **test plan** is a description of what JMeter should do when you press Run.

It answers three questions:

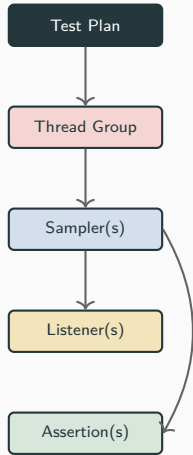
- **Who** is going to hit the server? (how many virtual users, arriving at what pace)
- **What** are they going to do? (which requests, in which order, with what data)
- **How** do we know what happened? (which results to collect, which conditions to check)

It is stored in a `.jmx` file (which is really just XML). You can commit it to Git, diff it, and be quietly disappointed with the diffs. It's XML.



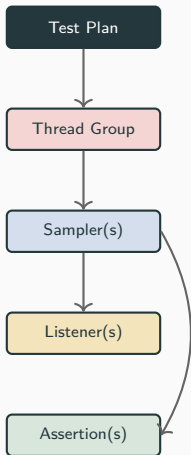
JMeter ships with **Templates** under `File` → `Templates` → `Building a Web Test Plan`, feel free to pay a visit.

The hierarchy: how the pieces fit

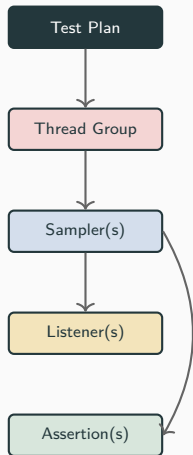


The hierarchy: how the pieces fit

- **Test Plan:** the root. Global settings live here.

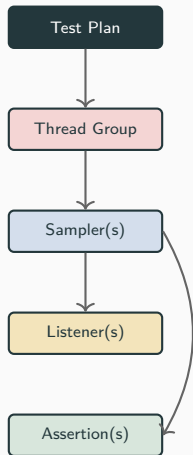


The hierarchy: how the pieces fit



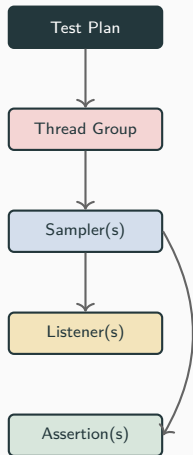
- **Test Plan:** the root. Global settings live here.
- **Thread Group:** “the users”. Says how many, how fast they arrive, and how many times each repeats.

The hierarchy: how the pieces fit



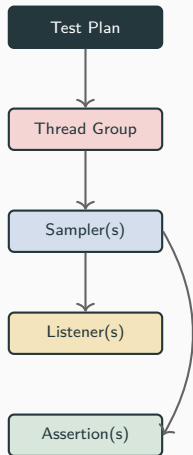
- **Test Plan:** the root. Global settings live here.
- **Thread Group:** “the users”. Says how many, how fast they arrive, and how many times each repeats.
- **Sampler:** an actual request. “Fetch /login.”

The hierarchy: how the pieces fit



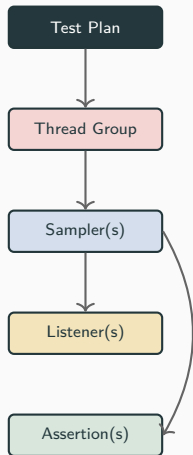
- **Test Plan:** the root. Global settings live here.
- **Thread Group:** “the users”. Says how many, how fast they arrive, and how many times each repeats.
- **Sampler:** an actual request. “Fetch /login.”
- **Listener:** catches results and shows them.

The hierarchy: how the pieces fit



- **Test Plan:** the root. Global settings live here.
- **Thread Group:** “the users”. Says how many, how fast they arrive, and how many times each repeats.
- **Sampler:** an actual request. “Fetch /login.”
- **Listener:** catches results and shows them.
- **Assertion:** a check on the response. “Was the status 200? Did the body contain ‘Welcome’?”

The hierarchy: how the pieces fit



- **Test Plan:** the root. Global settings live here.
- **Thread Group:** “the users”. Says how many, how fast they arrive, and how many times each repeats.
- **Sampler:** an actual request. “Fetch /login.”
- **Listener:** catches results and shows them.
- **Assertion:** a check on the response. “Was the status 200? Did the body contain ‘Welcome’?”

Right-click any node to Add its children. The tree grows sideways in the GUI, which nobody asked for, but here we are.

Building your first test plan

Our initial test plan.

1. **Start JMeter.** A blank Test Plan appears.

Building your first test plan

Our initial test plan.

1. **Start JMeter.** A blank Test Plan appears.
2. **Add a Thread Group.**

Right-click Test Plan → Add → Threads (Users) → Thread Group

Building your first test plan

Our initial test plan.

1. **Start JMeter.** A blank Test Plan appears.

2. **Add a Thread Group.**

Right-click Test Plan → Add → Threads (Users) → Thread Group

3. **Add an HTTP Request sampler.**

Right-click Thread Group → Add → Sampler → HTTP Request

Building your first test plan

Our initial test plan.

1. **Start JMeter.** A blank Test Plan appears.

2. **Add a Thread Group.**

Right-click Test Plan → Add → Threads (Users) → Thread Group

3. **Add an HTTP Request sampler.**

Right-click Thread Group → Add → Sampler → HTTP Request

4. **Conf. the req:** protocol: `https`, server: `jmeter.apache.org`, path: `/download_jmeter.cgi`.

Building your first test plan

Our initial test plan.

1. **Start JMeter.** A blank Test Plan appears.

2. **Add a Thread Group.**

Right-click Test Plan → Add → Threads (Users) → Thread Group

3. **Add an HTTP Request sampler.**

Right-click Thread Group → Add → Sampler → HTTP Request

4. **Conf. the req:** protocol: `https`, server: `jmeter.apache.org`, path: `/download_jmeter.cgi`.

5. **Add listeners so you can see results.**

Right-click Thread Group → Add → Listener → View Results Tree

(and add View Results in Table while you're there)

Building your first test plan

Our initial test plan.

1. **Start JMeter.** A blank Test Plan appears.

2. **Add a Thread Group.**

Right-click Test Plan → Add → Threads (Users) → Thread Group

3. **Add an HTTP Request sampler.**

Right-click Thread Group → Add → Sampler → HTTP Request

4. **Conf. the req:** protocol: `https`, server: `jmeter.apache.org`, path: `/download_jmeter.cgi`.

5. **Add listeners so you can see results.**

Right-click Thread Group → Add → Listener → View Results Tree

(and add View Results in Table while you're there)

6. **Save.** Then press the green triangle.

Building your first test plan

Our initial test plan.

1. **Start JMeter.** A blank Test Plan appears.

2. **Add a Thread Group.**

Right-click Test Plan → Add → Threads (Users) → Thread Group

3. **Add an HTTP Request sampler.**

Right-click Thread Group → Add → Sampler → HTTP Request

4. **Conf. the req:** protocol: `https`, server: `jmeter.apache.org`, path: `/download_jmeter.cgi`.

5. **Add listeners so you can see results.**

Right-click Thread Group → Add → Listener → View Results Tree

(and add View Results in Table while you're there)

6. **Save.** Then press the green triangle.



Save *before* you run. JMeter does not autosave. It also does not stop you from running a test that would embarrass your ISP. Both facts are, in principle, features.

Interlude: dissecting a web address

A URL, taken apart

Before we ask JMeter to send requests, we should know what a request *is*.

```
https://www.google.com/search?q=blazemeter&sca_esv=314db17b76bb14ae&source=hp
```

A URL, taken apart

Before we ask JMeter to send requests, we should know what a request *is*.

`https://www.google.com/search?q=blazemeter&sca_esv=314db17b76bb14ae&source=hp`

- **Protocol:** https — how to speak

A URL, taken apart

Before we ask JMeter to send requests, we should know what a request *is*.

```
https://www.google.com/search?q=blazemeter&sca_esv=314db17b76bb14ae&source=hp
```

- **Protocol:** https — how to speak
- **Server (host):** www.google.com — who to speak to

A URL, taken apart

Before we ask JMeter to send requests, we should know what a request *is*.

```
https://www.google.com/search?q=blazemeter&sca_esv=314db17b76bb14ae&source=hp
```

- **Protocol:** https — how to speak
- **Server (host):** www.google.com — who to speak to
- **Path:** /search — which endpoint on that server

A URL, taken apart

Before we ask JMeter to send requests, we should know what a request *is*.

```
https://www.google.com/search?q=blazemeter&sca_esv=314db17b76bb14ae&source=hp
```

- **Protocol:** https — how to speak
- **Server (host):** www.google.com — who to speak to
- **Path:** /search — which endpoint on that server
- **Parameters:** everything after ?, separated by &
key-value pairs the server uses to decide what to send back

A URL, taken apart

Before we ask JMeter to send requests, we should know what a request *is*.

```
https://www.google.com/search?q=blazemeter&sca_esv=314db17b76bb14ae&source=hp
```

- **Protocol:** https — how to speak
- **Server (host):** www.google.com — who to speak to
- **Path:** /search — which endpoint on that server
- **Parameters:** everything after ?, separated by &
key-value pairs the server uses to decide what to send back



When you configure an HTTP Request sampler in JMeter, you are filling in *exactly* these fields. If you can read a URL, you can configure a sampler. That is the entire trick.

Anatomy of an HTTP Request sampler

Basic fields (top of the panel):

- **Protocol:** http or https (leave blank $\Rightarrow$ http)
- **Server Name or IP:** `jmeter.apache.org`
Without the scheme, without a slash, just the host.
- **Port:** usually blank (443 for https, 80 for http)
- **Method:** GET, POST, PUT, DELETE, ...
- **Path:** e.g. `/notices/`

Then, one of these tabs:

- **Parameters:** for query strings or form fields
- **Body Data:** for raw JSON / XML payloads
- **Files Upload:** for multipart uploads

The image shows a screenshot of the 'HTTP Request' configuration panel in Apache JMeter. The panel has a title 'HTTP Request' and several input fields: 'Protocol' (set to 'https'), 'Server' (set to 'jmeter.apache.org'), 'Port' (empty), 'Method' (set to 'GET'), and 'Path' (set to '/'). Below these fields are three tabs: 'Parameters', 'Body Data', and 'Files Upload'. The 'Parameters' tab is selected, and its content area contains the text '(key/value rows for query string or form)'.

HTTP Request	
Protocol	https
Server	jmeter.apache.org
Port	
Method	GET
Path	/
Parameters Body Data Files Upload	
(key/value rows for query string or form)	

Thread groups: simulating users

Thread groups: “how many, how fast, how often”

A **thread group** answers three questions with three little numbers.

Thread Group

Number of Threads (users):

50

Ramp-up period (seconds):

10

Loop Count:

5

or check Infinite, or set a *Duration*

Thread groups: “how many, how fast, how often”

A **thread group** answers three questions with three little numbers.

Thread Group

Number of Threads (users):

50

→ how many virtual users

Ramp-up period (seconds):

10

Loop Count:

5

or check Infinite, or set a *Duration*

Thread groups: “how many, how fast, how often”

A **thread group** answers three questions with three little numbers.

Thread Group

Number of Threads (users):

50

→ how many virtual users

Ramp-up period (seconds):

10

→ how quickly they arrive

Loop Count:

5

or check Infinite, or set a *Duration*

Thread groups: “how many, how fast, how often”

A **thread group** answers three questions with three little numbers.

Thread Group

Number of Threads (users):

50

→ how many virtual users

Ramp-up period (seconds):

10

→ how quickly they arrive

Loop Count:

5

or check Infinite, or set a *Duration*

→ how many times each repeats

Thread groups: “how many, how fast, how often”

A **thread group** answers three questions with three little numbers.

Thread Group		
Number of Threads (users):	50	→ how many virtual users
Ramp-up period (seconds):	10	→ how quickly they arrive
Loop Count:	5	→ or check Infinite, or set a <i>Duration</i> → how many times each repeats

In JMeter, each “thread” is one virtual user. JMeter really does spin up a Java thread for each one. This is a common source of confusion, and the reason you should not set 10,000 threads on a regular machine.

Ramp-up: what it *actually* means

Ramp-up = the time JMeter takes to start *all* the threads, spaced evenly.

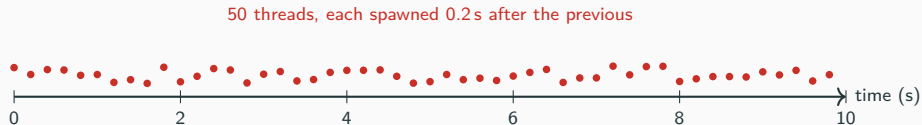
With **50 threads** and a **10-second ramp-up**, JMeter starts a new thread every $10/50 = 0.2$ seconds.
All 50 are running by the 10-second mark.



Ramp-up: what it *actually* means

Ramp-up = the time JMeter takes to start *all* the threads, spaced evenly.

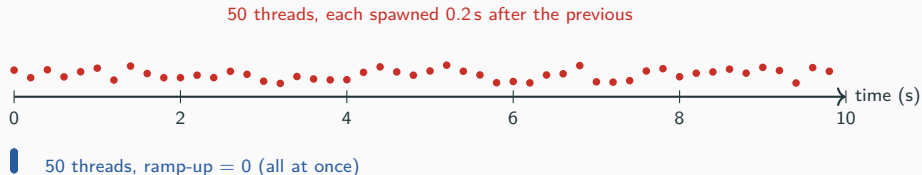
With **50 threads** and a **10-second ramp-up**, JMeter starts a new thread every $10/50 = 0.2$ seconds. All 50 are running by the 10-second mark.



Ramp-up: what it *actually* means

Ramp-up = the time JMeter takes to start *all* the threads, spaced evenly.

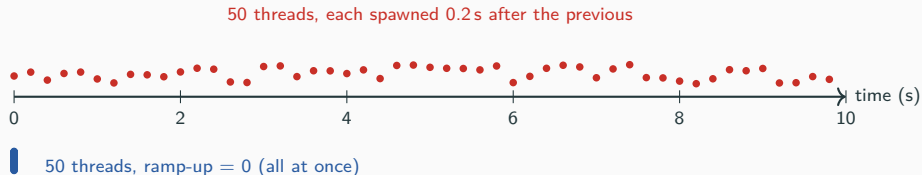
With **50 threads** and a **10-second ramp-up**, JMeter starts a new thread every $10/50 = 0.2$ seconds. All 50 are running by the 10-second mark.



Ramp-up: what it *actually* means

Ramp-up = the time JMeter takes to start *all* the threads, spaced evenly.

With **50 threads** and a **10-second ramp-up**, JMeter starts a new thread every $10/50 = 0.2$ seconds. All 50 are running by the 10-second mark.



Ramp-up of **0** simulates every user arriving at the same instant — realistic only during a DDoS. Pick a ramp-up long enough that setup cost doesn't dominate the first samples, but short enough that you reach steady state during the run.

Loop count vs. Duration

Two ways to say “how long should the test go on for.”

Loop Count

Each thread repeats the entire request sequence this many times, then dies.

- Predictable *amount of work*.
- Total test time varies with response time.

Scheduler / Duration

Threads keep looping until a wall-clock deadline.

- Predictable *amount of time*.
- Enable the scheduler and set `Duration` in seconds.

Loop count vs. Duration

Two ways to say “how long should the test go on for.”

Loop Count

Each thread repeats the entire request sequence this many times, then dies.

- Predictable *amount of work*.
- Total test time varies with response time.

Scheduler / Duration

Threads keep looping until a wall-clock deadline.

- Predictable *amount of time*.
- Enable the scheduler and set `Duration` in seconds.

? Why do we care?

Load tests are about *steady-state behaviour*. Very short runs measure warm-up, not performance.

**Samplers: the things that
actually do things**

Samplers: the “do a thing” step

A **sampler** is a single unit of work sent by a virtual user. For a web app, that unit is almost always an **HTTP Request**.

Each sampler:



*measure the time
between these two*

Samplers: the “do a thing” step

A **sampler** is a single unit of work sent by a virtual user. For a web app, that unit is almost always an **HTTP Request**.

Each sampler:

- **sends** a request to some server,



*measure the time
between these two*

Samplers: the “do a thing” step

A **sampler** is a single unit of work sent by a virtual user. For a web app, that unit is almost always an **HTTP Request**.

Each sampler:

- **sends** a request to some server,
- **waits** for the response,



*measure the time
between these two*

Samplers: the “do a thing” step

A **sampler** is a single unit of work sent by a virtual user. For a web app, that unit is almost always an **HTTP Request**.

Each sampler:

- **sends** a request to some server,
- **waits** for the response,
- **measures** how long that took, and



*measure the time
between these two*

Samplers: the “do a thing” step

A **sampler** is a single unit of work sent by a virtual user. For a web app, that unit is almost always an **HTTP Request**.

Each sampler:

- **sends** a request to some server,
- **waits** for the response,
- **measures** how long that took, and
- **hands** the result over to any listeners and assertions attached to it.



*measure the time
between these two*

Samplers: the “do a thing” step

A **sampler** is a single unit of work sent by a virtual user. For a web app, that unit is almost always an **HTTP Request**.

Each sampler:

- **sends** a request to some server,
- **waits** for the response,
- **measures** how long that took, and
- **hands** the result over to any listeners and assertions attached to it.

Everything else in the test plan — thread groups, listeners, config elements — exists to *support* samplers.



*measure the time
between these two*

Adding a sampler

1. **Right-click your Thread Group** → Add → Sampler → HTTP Request.
2. **Fill in** protocol, server, path, method (the URL bits we dissected earlier).
3. **Rename it** to something readable — e.g. “Login POST”, “Home page GET”.
(Left panel names show up verbatim in every report. Future-you will thank present-you.)

You probably don't need most of these, but they exist:

- **Debug Sampler** — dumps JMeter variables into results. Priceless during recording debugging, useless in production runs.
- **BeanShell / JSR223 Sampler** — run arbitrary Groovy / Java code.
- **FTP, JDBC, JUnit, LDAP, Mail Reader, TCP Sampler** — for their respective protocols.

Available samplers (partial)

- HTTP Request
- Debug Sampler
- BeanShell / JSR223
- FTP Request
- JDBC Request
- JUnit Request
- Java Request
- LDAP Request
- Mail Reader Sampler

Listeners: seeing what happened

Listeners: the reporters

A **listener** watches samplers finish and does something with the result: writes it to a file, draws a graph, updates a table.

Listeners: the reporters

A **listener** watches samplers finish and does something with the result: writes it to a file, draws a graph, updates a table.

- **View Results Tree** — shows every request and response in full, with headers, body, and timing. Indispensable while building a test.
(Do not leave it on during a load test. It stores everything in RAM.)

Listeners: the reporters

A **listener** watches samplers finish and does something with the result: writes it to a file, draws a graph, updates a table.

- **View Results Tree** — shows every request and response in full, with headers, body, and timing. Indispensable while building a test.
(Do not leave it on during a load test. It stores everything in RAM.)
- **View Results in Table** — one row per sample, with time, latency, status.

Listeners: the reporters

A **listener** watches samplers finish and does something with the result: writes it to a file, draws a graph, updates a table.

- **View Results Tree** — shows every request and response in full, with headers, body, and timing. Indispensable while building a test.
(Do not leave it on during a load test. It stores everything in RAM.)
- **View Results in Table** — one row per sample, with time, latency, status.
- **Summary Report** — aggregate stats: count, average, min, max, error %, throughput.

Listeners: the reporters

A **listener** watches samplers finish and does something with the result: writes it to a file, draws a graph, updates a table.

- **View Results Tree** — shows every request and response in full, with headers, body, and timing. Indispensable while building a test.
(Do not leave it on during a load test. It stores everything in RAM.)
- **View Results in Table** — one row per sample, with time, latency, status.
- **Summary Report** — aggregate stats: count, average, min, max, error %, throughput.
- **Aggregate Report** — like Summary Report, but with percentiles (90th, 95th, 99th).
The 99th percentile is usually where the ugly stories live.

Listeners: the reporters

A **listener** watches samplers finish and does something with the result: writes it to a file, draws a graph, updates a table.

- **View Results Tree** — shows every request and response in full, with headers, body, and timing. Indispensable while building a test.
(Do not leave it on during a load test. It stores everything in RAM.)
- **View Results in Table** — one row per sample, with time, latency, status.
- **Summary Report** — aggregate stats: count, average, min, max, error %, throughput.
- **Aggregate Report** — like Summary Report, but with percentiles (90th, 95th, 99th).
The 99th percentile is usually where the ugly stories live.
- **Graph Results** — pretty; not useful for making decisions.

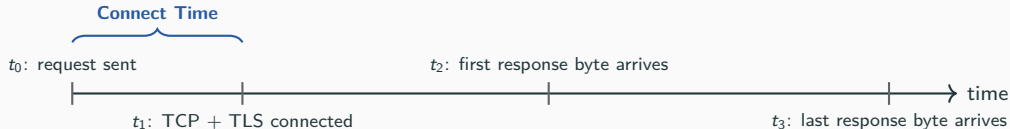
Listeners: the reporters

A **listener** watches samplers finish and does something with the result: writes it to a file, draws a graph, updates a table.

- **View Results Tree** — shows every request and response in full, with headers, body, and timing. Indispensable while building a test.
(Do not leave it on during a load test. It stores everything in RAM.)
- **View Results in Table** — one row per sample, with time, latency, status.
- **Summary Report** — aggregate stats: count, average, min, max, error %, throughput.
- **Aggregate Report** — like Summary Report, but with percentiles (90th, 95th, 99th).
The 99th percentile is usually where the ugly stories live.
- **Graph Results** — pretty; not useful for making decisions.
- **Simple Data Writer** — writes raw samples to a file, cheaply.
This is the one you keep enabled during actual runs.

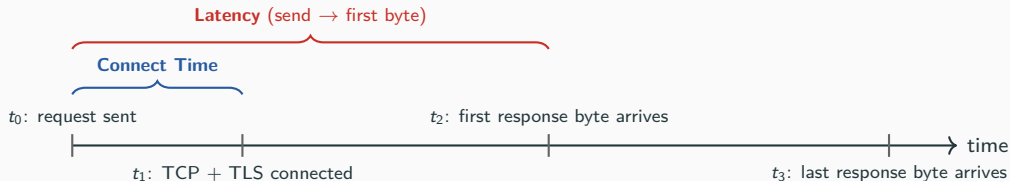
Timing terminology

When a listener says “latency” and “sample time”, it means specific things.



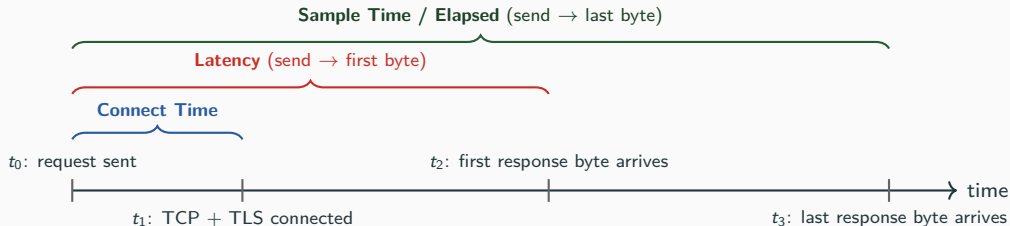
Timing terminology

When a listener says “latency” and “sample time”, it means specific things.



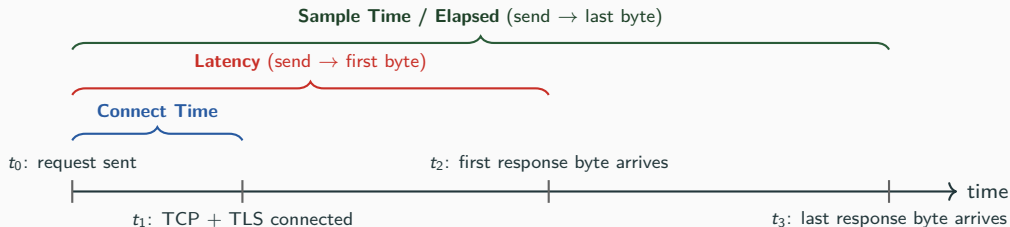
Timing terminology

When a listener says “latency” and “sample time”, it means specific things.



Timing terminology

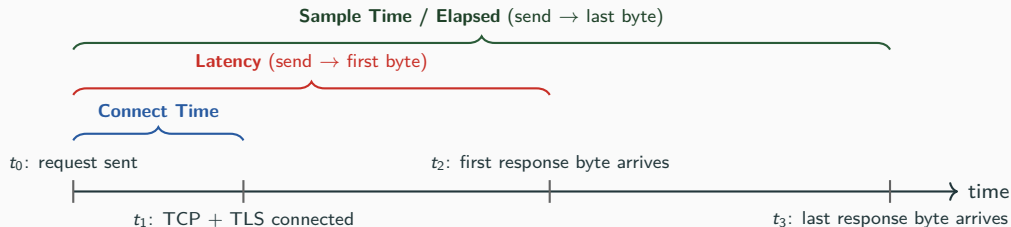
When a listener says “latency” and “sample time”, it means specific things.



- **Connect Time:** TCP handshake + TLS negotiation (if HTTPS).
- **Latency:** send-last-byte → receive-first-byte. Roughly, “server think time + network round trip”.
- **Sample Time (a.k.a. Elapsed):** the total, end to end..

Timing terminology

When a listener says “latency” and “sample time”, it means specific things.



- **Connect Time:** TCP handshake + TLS negotiation (if HTTPS).
- **Latency:** send-last-byte $\rightarrow$ receive-first-byte. Roughly, “server think time + network round trip”.
- **Sample Time** (a.k.a. **Elapsed**): the total, end to end..



Latency $\neq$ response time. A 20 KB response over a slow link can have low latency and large sample time.



Listeners are expensive. View Results Tree in particular keeps every request and response in memory. During a real load test, this will:

- slow JMeter down (skewing your timing measurements),
- eat gigabytes of RAM, and
- eventually crash the JVM with a satisfying `OutOfMemoryError`.

Practical rule:



Listeners are expensive. View Results Tree in particular keeps every request and response in memory. During a real load test, this will:

- slow JMeter down (skewing your timing measurements),
- eat gigabytes of RAM, and
- eventually crash the JVM with a satisfying `OutOfMemoryError`.

Practical rule:

- While *building* the test: View Results Tree on. Debug freely.



Listeners are expensive. View Results Tree in particular keeps every request and response in memory. During a real load test, this will:

- slow JMeter down (skewing your timing measurements),
- eat gigabytes of RAM, and
- eventually crash the JVM with a satisfying `OutOfMemoryError`.

Practical rule:

- While *building* the test: View Results Tree on. Debug freely.
- While *running* the test: only Simple Data Writer (writes CSV/JTL [JMeter Test Log] to disk).



Listeners are expensive. View Results Tree in particular keeps every request and response in memory. During a real load test, this will:

- slow JMeter down (skewing your timing measurements),
- eat gigabytes of RAM, and
- eventually crash the JVM with a satisfying `OutOfMemoryError`.

Practical rule:

- While *building* the test: View Results Tree on. Debug freely.
- While *running* the test: only Simple Data Writer (writes CSV/JTL [JMeter Test Log] to disk).
- After the test: open the JTL file in a fresh JMeter GUI, or generate the HTML report from the CLI.

**Assertions: was the response
actually correct?**

Assertions: because “200 OK” does not mean “correct”

A server can return **200 OK** with an error page in the body, or the login page instead of the dashboard. JMeter, by default, counts both as successes.

Assertions: because “200 OK” does not mean “correct”

A server can return **200 OK** with an error page in the body, or the login page instead of the dashboard. JMeter, by default, counts both as successes. An **assertion** checks the response against a rule and marks the sample failed if it doesn't match.

Assertions: because “200 OK” does not mean “correct”

A server can return **200 OK** with an error page in the body, or the login page instead of the dashboard. JMeter, by default, counts both as successes. An **assertion** checks the response against a rule and marks the sample failed if it doesn't match.

Common types

- **Response Assertion** — “body contains Welcome, \$user”
- **Duration Assertion** — “response < 500 ms”
- **Size Assertion** — “response 1–10 KB”
- **JSON / XPath Assertion** — match via JSONPath / XPath
- **HTML Assertion** — well-formedness via Tidy

Assertions: because “200 OK” does not mean “correct”

A server can return **200 OK** with an error page in the body, or the login page instead of the dashboard. JMeter, by default, counts both as successes. An **assertion** checks the response against a rule and marks the sample failed if it doesn't match.

Common types

- **Response Assertion** — “body contains Welcome, \$user”
- **Duration Assertion** — “response < 500 ms”
- **Size Assertion** — “response 1–10 KB”
- **JSON / XPath Assertion** — match via JSONPath / XPath
- **HTML Assertion** — well-formedness via Tidy

Where to put them

- Under a **sampler**: applies to that one.
- Under the **thread group**: applies to every sampler in it.
- Under the **test plan**: applies to everything.

The narrower the scope, the more surgical (and less surprising) the failures.

Assertions: because “200 OK” does not mean “correct”

A server can return **200 OK** with an error page in the body, or the login page instead of the dashboard. JMeter, by default, counts both as successes. An **assertion** checks the response against a rule and marks the sample failed if it doesn't match.

Common types

- **Response Assertion** — “body contains Welcome, \$user”
- **Duration Assertion** — “response < 500 ms”
- **Size Assertion** — “response 1–10 KB”
- **JSON / XPath Assertion** — match via JSONPath / XPath
- **HTML Assertion** — well-formedness via Tidy

Where to put them

- Under a **sampler**: applies to that one.
- Under the **thread group**: applies to every sampler in it.
- Under the **test plan**: applies to everything.

The narrower the scope, the more surgical (and less surprising) the failures.

? Why do we care?

Without assertions, a load test tells you the server *responded*, not that it *worked*.

**Recording: because typing
HTTP by hand is a punishment**

Why record?

A real user journey (log in, browse, add to cart, check out) can be 40+ HTTP requests with cookies, tokens, redirects and AJAX calls. Typing all of that into HTTP samplers by hand is:

Why record?

A real user journey (log in, browse, add to cart, check out) can be 40+ HTTP requests with cookies, tokens, redirects and AJAX calls. Typing all of that into HTTP samplers by hand is:

- tedious,

Why record?

A real user journey (log in, browse, add to cart, check out) can be 40+ HTTP requests with cookies, tokens, redirects and AJAX calls. Typing all of that into HTTP samplers by hand is:

- tedious,
- error-prone,

Why record?

A real user journey (log in, browse, add to cart, check out) can be 40+ HTTP requests with cookies, tokens, redirects and AJAX calls. Typing all of that into HTTP samplers by hand is:

- tedious,
- error-prone,
- and by the time you're done, the front-end team has changed the URLs.

Why record?

A real user journey (log in, browse, add to cart, check out) can be 40+ HTTP requests with cookies, tokens, redirects and AJAX calls. Typing all of that into HTTP samplers by hand is:

- tedious,
- error-prone,
- and by the time you're done, the front-end team has changed the URLs.

Instead: **record** the journey by pointing your browser at JMeter, doing the thing once by hand, and letting JMeter turn every request it sees into a sampler.

Why record?

A real user journey (log in, browse, add to cart, check out) can be 40+ HTTP requests with cookies, tokens, redirects and AJAX calls. Typing all of that into HTTP samplers by hand is:

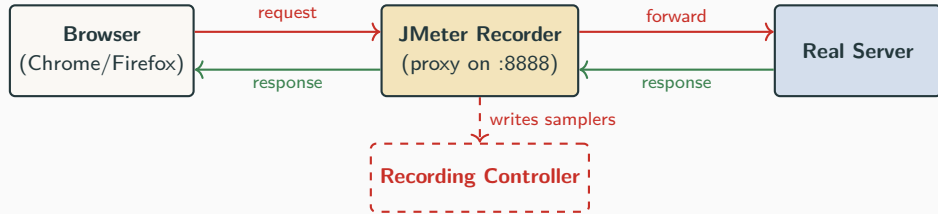
- tedious,
- error-prone,
- and by the time you're done, the front-end team has changed the URLs.

Instead: **record** the journey by pointing your browser at JMeter, doing the thing once by hand, and letting JMeter turn every request it sees into a sampler.

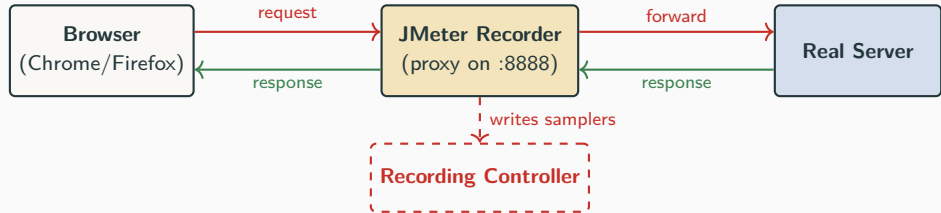
❓ Why do we care?

JMeter's recorder is a **man-in-the-middle proxy**. Your browser talks to JMeter as if it were the internet; JMeter forwards everything to the real internet, writes down what passed through, and hands the responses back.

The proxy architecture



The proxy architecture

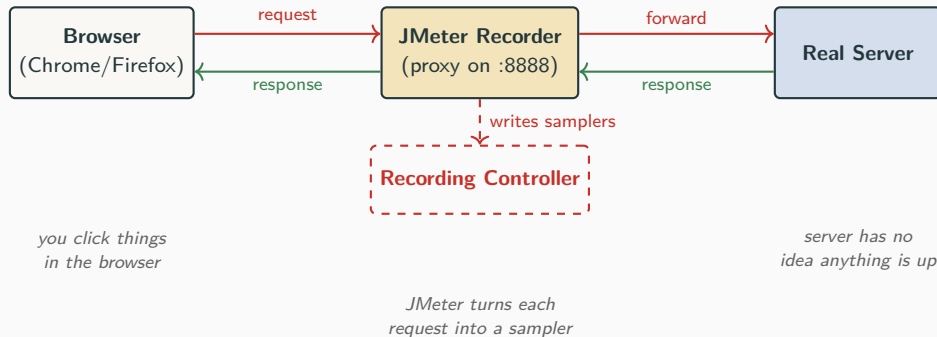


*you click things
in the browser*

*server has no
idea anything is up*

*JMeter turns each
request into a sampler*

The proxy architecture



For HTTPS to work, JMeter generates its own certificate that your browser must trust. Otherwise the browser — reasonably — refuses to forward encrypted traffic through a strange proxy.

Setting up recording, in order

1. **Add a Recording Controller** under your Thread Group.

Right-click Thread Group → Add → Logic Controller → Recording Controller.

Recorded samplers will be dropped in here. You can add several (rename them “login”, “search”, “checkout”).

Setting up recording, in order

1. **Add a Recording Controller** under your Thread Group.

Right-click Thread Group → Add → Logic Controller → Recording Controller.

Recorded samplers will be dropped in here. You can add several (rename them “login”, “search”, “checkout”).

2. **Add the HTTP(S) Test Script Recorder** to the Test Plan.

Test Plan → Add → Non-Test Elements → HTTP(S) Test Script Recorder.

Set the port (8888 is traditional), and set *Target Controller* to point at your Recording Controller.

Setting up recording, in order

1. **Add a Recording Controller** under your Thread Group.

Right-click Thread Group → Add → Logic Controller → Recording Controller.

Recorded samplers will be dropped in here. You can add several (rename them “login”, “search”, “checkout”).

2. **Add the HTTP(S) Test Script Recorder** to the Test Plan.

Test Plan → Add → Non-Test Elements → HTTP(S) Test Script Recorder.

Set the port (8888 is traditional), and set *Target Controller* to point at your Recording Controller.

3. **Start the recorder once** (Start button in the recorder panel) so that JMeter generates its `ApacheJMeterTemporaryRootCA.crt` into `jmeter/bin/`. Then Stop.

Setting up recording, in order

1. **Add a Recording Controller** under your Thread Group.

Right-click Thread Group → Add → Logic Controller → Recording Controller.

Recorded samplers will be dropped in here. You can add several (rename them “login”, “search”, “checkout”).

2. **Add the HTTP(S) Test Script Recorder** to the Test Plan.

Test Plan → Add → Non-Test Elements → HTTP(S) Test Script Recorder.

Set the port (8888 is traditional), and set *Target Controller* to point at your Recording Controller.

3. **Start the recorder once** (Start button in the recorder panel) so that JMeter generates its `ApacheJMeterTemporaryRootCA.crt` into `jmeter/bin/`. Then Stop.
4. **Install that certificate** in your browser as a trusted authority.

Chrome: Settings → Privacy & Security → Security → Manage certificates → Authorities → Import.

Setting up recording, in order

1. **Add a Recording Controller** under your Thread Group.

Right-click Thread Group → Add → Logic Controller → Recording Controller.

Recorded samplers will be dropped in here. You can add several (rename them “login”, “search”, “checkout”).

2. **Add the HTTP(S) Test Script Recorder** to the Test Plan.

Test Plan → Add → Non-Test Elements → HTTP(S) Test Script Recorder.

Set the port (8888 is traditional), and set *Target Controller* to point at your Recording Controller.

3. **Start the recorder once** (Start button in the recorder panel) so that JMeter generates its `ApacheJMeterTemporaryRootCA.crt` into `jmeter/bin/`. Then Stop.

4. **Install that certificate** in your browser as a trusted authority.

Chrome: Settings → Privacy & Security → Security → Manage certificates → Authorities → Import.

5. **Point your browser's proxy** at `localhost:8888`.

Setting up recording, in order

1. **Add a Recording Controller** under your Thread Group.

Right-click Thread Group → Add → Logic Controller → Recording Controller.

Recorded samplers will be dropped in here. You can add several (rename them “login”, “search”, “checkout”).

2. **Add the HTTP(S) Test Script Recorder** to the Test Plan.

Test Plan → Add → Non-Test Elements → HTTP(S) Test Script Recorder.

Set the port (8888 is traditional), and set *Target Controller* to point at your Recording Controller.

3. **Start the recorder once** (Start button in the recorder panel) so that JMeter generates its `ApacheJMeterTemporaryRootCA.crt` into `jmeter/bin/`. Then Stop.

4. **Install that certificate** in your browser as a trusted authority.

Chrome: Settings → Privacy & Security → Security → Manage certificates → Authorities → Import.

5. **Point your browser's proxy** at `localhost:8888`.

6. **Start the recorder**, browse the site, then **Stop** it.

The Recording Controller now holds one sampler per request the browser made.

Order matters.



Certificate first, proxy second. If you set the proxy before installing the certificate, HTTPS pages will fail to load, and the internet will — correctly — appear to be broken.



After you record, undo the browser proxy. Otherwise your regular browsing goes through a JMeter that isn't running, and the internet stops working again. This will eventually happen to you.

A few more tips

- **Filter out noise.** In the recorder, add URL exclusion patterns for `.*\.(png|jpg|gif|css|js|woff|ico)` unless you specifically want to test static asset delivery.

A few more tips

- **Filter out noise.** In the recorder, add URL exclusion patterns for `.*\.(png|jpg|gif|css|js|woff|ico)` unless you specifically want to test static asset delivery.
- **Chrome is stricter about certificates than Firefox.** If Chrome keeps refusing, Firefox is a fine second choice.

A few more tips

- **Filter out noise.** In the recorder, add URL exclusion patterns for `.*\.(png|jpg|gif|css|js|woff|ico)` unless you specifically want to test static asset delivery.
- **Chrome is stricter about certificates than Firefox.** If Chrome keeps refusing, Firefox is a fine second choice.
- **Try the “Recording” template.** File → Templates → Recording pre-builds most of the above. It's the polite thing to do.

A few more tips

- **Filter out noise.** In the recorder, add URL exclusion patterns for `.*\.(png|jpg|gif|css|js|woff|ico)` unless you specifically want to test static asset delivery.
- **Chrome is stricter about certificates than Firefox.** If Chrome keeps refusing, Firefox is a fine second choice.
- **Try the “Recording” template.** File → Templates → Recording pre-builds most of the above. It’s the polite thing to do.
- **Try recording against a demo site** first, e.g. `opensource-demo.orangehrmlive.com`. Recording your own site while learning is a way to conflate two problems.

After you record: clean up

A raw recording is not a good test. It is a starting point.

- **Rename samplers** to describe what the user was doing — “Open login page”, “Submit credentials”, “Fetch dashboard”. The auto-names are URLs and are unreadable in reports.

After you record: clean up

A raw recording is not a good test. It is a starting point.

- **Rename samplers** to describe what the user was doing — “Open login page”, “Submit credentials”, “Fetch dashboard”. The auto-names are URLs and are unreadable in reports.
- **Extract server-generated values** (CSRF tokens, session IDs) using *Post-Processors* (Regular Expression Extractor, CSS Selector Extractor, JSON Extractor) and reference them in later requests as `${tokenName}`.

After you record: clean up

A raw recording is not a good test. It is a starting point.

- **Rename samplers** to describe what the user was doing — “Open login page”, “Submit credentials”, “Fetch dashboard”. The auto-names are URLs and are unreadable in reports.
- **Extract server-generated values** (CSRF tokens, session IDs) using *Post-Processors* (Regular Expression Extractor, CSS Selector Extractor, JSON Extractor) and reference them in later requests as `${tokenName}`.
- **Add think time** with Timers, so you're not hammering the server faster than any real user would.

After you record: clean up

A raw recording is not a good test. It is a starting point.

- **Rename samplers** to describe what the user was doing — “Open login page”, “Submit credentials”, “Fetch dashboard”. The auto-names are URLs and are unreadable in reports.
- **Extract server-generated values** (CSRF tokens, session IDs) using *Post-Processors* (Regular Expression Extractor, CSS Selector Extractor, JSON Extractor) and reference them in later requests as `${tokenName}`.
- **Add think time** with Timers, so you're not hammering the server faster than any real user would.
- **Add assertions** on the important responses so a broken workflow shows up as a failure, not a shrug.

After you record: clean up

A raw recording is not a good test. It is a starting point.

- **Rename samplers** to describe what the user was doing — “Open login page”, “Submit credentials”, “Fetch dashboard”. The auto-names are URLs and are unreadable in reports.
- **Extract server-generated values** (CSRF tokens, session IDs) using *Post-Processors* (Regular Expression Extractor, CSS Selector Extractor, JSON Extractor) and reference them in later requests as `${tokenName}`.
- **Add think time** with Timers, so you're not hammering the server faster than any real user would.
- **Add assertions** on the important responses so a broken workflow shows up as a failure, not a shrug.
- **Add Listeners** (Summary / Aggregate Report + Simple Data Writer) and run.

But how does browsing become load testing?

A fair question: if the recorder just captured one Chrome window doing one journey, how do we get *load*?

But how does browsing become load testing?

A fair question: if the recorder just captured one Chrome window doing one journey, how do we get *load*?

Three moves:

- **Record once.** You, one browser, one journey $\Rightarrow$ a list of samplers in the Recording Controller.

But how does browsing become load testing?

A fair question: if the recorder just captured one Chrome window doing one journey, how do we get *load*?

Three moves:

- **Record once.** You, one browser, one journey $\Rightarrow$ a list of samplers in the Recording Controller.
- **Multiply with the Thread Group.** Threads=10, Loops=5 tells JMeter to replay that same list **50 times**, spread across **10 parallel workers**.

But how does browsing become load testing?

A fair question: if the recorder just captured one Chrome window doing one journey, how do we get *load*?

Three moves:

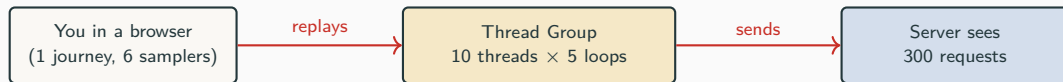
- **Record once.** You, one browser, one journey $\Rightarrow$ a list of samplers in the Recording Controller.
- **Multiply with the Thread Group.** Threads=10, Loops=5 tells JMeter to replay that same list **50 times**, spread across **10 parallel workers**.
- **Isolate with the Cookie Manager.** Each thread gets its own cookie jar $\Rightarrow$ ten independent logins, not ten requests sharing one session.

But how does browsing become load testing?

A fair question: if the recorder just captured one Chrome window doing one journey, how do we get *load*?

Three moves:

- **Record once.** You, one browser, one journey $\Rightarrow$ a list of samplers in the Recording Controller.
- **Multiply with the Thread Group.** Threads=10, Loops=5 tells JMeter to replay that same list **50 times**, spread across **10 parallel workers**.
- **Isolate with the Cookie Manager.** Each thread gets its own cookie jar $\Rightarrow$ ten independent logins, not ten requests sharing one session.



The command line: for grown-ups

Why the CLI?

The GUI is friendly. It is also:

Why the CLI?

The GUI is friendly. It is also:

- a memory hog (Your listeners keep everything in RAM),

Why the CLI?

The GUI is friendly. It is also:

- a memory hog (Your listeners keep everything in RAM),
- slower than the CLI on the same test plan,

Why the CLI?

The GUI is friendly. It is also:

- a memory hog (Your listeners keep everything in RAM),
- slower than the CLI on the same test plan,
- incapable of feeding results into a CI/CD pipeline, and
- officially not recommended for load runs (JMeter itself tells you so at startup).

Why the CLI?

The GUI is friendly. It is also:

- a memory hog (Your listeners keep everything in RAM),
- slower than the CLI on the same test plan,
- incapable of feeding results into a CI/CD pipeline, and
- officially not recommended for load runs (JMeter itself tells you so at startup).

The workflow is:

Build the `.jmx` in the GUI → Run it from the CLI → Open the results back in the GUI (or as HTML).

Why the CLI?

The GUI is friendly. It is also:

- a memory hog (Your listeners keep everything in RAM),
- slower than the CLI on the same test plan,
- incapable of feeding results into a CI/CD pipeline, and
- officially not recommended for load runs (JMeter itself tells you so at startup).

The workflow is:

Build the `.jmx` in the GUI → Run it from the CLI → Open the results back in the GUI (or as HTML).



Confusing “the GUI is for building” and “the CLI is for running” is a rite of passage. You will do it once, watch a test plan crawl, and never do it again.

The CLI command, in one line

From the JMeter bin/ folder (or with bin/ on your PATH):

```
jmeter -n -t path/to/TestPlan.jmx -l path/to/results.jtl \  
-e -o path/to/html-report/
```

flag	meaning
-n	non-GUI mode (this is the point)
-t	test plan file (.jmx)
-l	log results to a file (.jtl, essentially CSV)
-e	at the end, generate the HTML report
-o	output folder for the HTML report (must not already exist)
-g	generate the HTML report from an existing .jtl, without re-running
-R	distributed: list of Remote hosts to run on
-h	help, the polite way to remember flags

Two everyday CLI recipes

1. Basic load test, run and reported in one go:

```
jmeter -n -t OrangeHRM.jmx -l results.jtl -e -o report/
```

Two everyday CLI recipes

1. Basic load test, run and reported in one go:

```
jmeter -n -t OrangeHRM.jmx -l results.jtl -e -o report/
```

2. Override thread count and duration from the command line:

```
jmeter -n -t OrangeHRM.jmx -l results.jtl \  
-Jthreads=50 -Jduration=300
```

3. Generate the HTML report from a previous run's raw results:

```
jmeter -g results.jtl -o report/
```

Useful when you kept the raw data but discarded the report.

3. Generate the HTML report from a previous run's raw results:

```
jmeter -g results.jtl -o report/
```

Useful when you kept the raw data but discarded the report.

4. Distributed test, driving load from several machines:

```
jmeter -n -t OrangeHRM.jmx -r -R server1,server2 -l master.jtl
```

`-r` tells the local “controller” to trigger every remote “worker” listed by `-R`. Each worker needs `jmeter-server` running and reachable on the network. This is how you generate load a single laptop couldn't sustain — five machines of 200 users beats one machine of 1000.

Putting it all together: an OrangeHRM walkthrough

Case study: load-testing the OrangeHRM demo

Target: `opensource-demo.orangehrmlive.com` — a public demo of an HR system. Login is Admin / admin123.

The journey each user performs

1. Open the login page
2. Submit Admin / admin123
3. Land on the dashboard
4. Click **Admin** in the left nav
5. Click **PIM** in the left nav
6. Click profile picture → **Logout**

The load profile

Threads (users):	10
Ramp-up:	10 seconds
Loop count:	5
Total journeys:	50
Total HTTP requests (approx):	$300 \times 50 \times 6$
User arrival rate:	1 per second

Walkthrough — step 1: record the journey

With the recorder set up (Recording Controller, HTTP(S) Test Script Recorder on 8888, browser proxied, certificate installed):

1. Press **Start** on the recorder panel.
2. In Chrome, open `opensource-demo.orangehrmlive.com/web/index.php/auth/login`.
3. Type Admin / admin123, click **Login**.
4. On the dashboard, click **Admin** in the left nav, then **PIM**.
5. Click the profile picture (top right) → **Logout**.
6. Press **Stop** on the recorder panel.

What's now inside the Recording Controller (a lot filtered out):

```
Recording Controller
|-- GET    /web/index.php/auth/login
|-- POST   /web/index.php/auth/validate
|-- GET    /web/index.php/dashboard/index
|-- GET    /web/index.php/admin/viewSystemUsers
|-- GET    /web/index.php/pim/viewEmployeeList
`-- GET    /web/index.php/auth/logout
```

Walkthrough — step 2: clean up + add assertions

Three fixes turn the raw recording into a real test:

- **Extract the CSRF token.** OrangeHRM's login POST includes a `_token` field, freshly generated on each GET of `/auth/login`. Add a *Regular Expression Extractor* as a child of the login-page sampler (Sampler → Add → Post-Processor → Regular Expression Extractor). Fill in:

Name of variable: `loginToken` Regex: `name="_token" value="([^\"]+)"` Template: `$1$`

Then in the login POST body, replace the recorded token value with `${loginToken}`.

Walkthrough — step 2: clean up + add assertions

Three fixes turn the raw recording into a real test:

- **Extract the CSRF token.** OrangeHRM's login POST includes a `_token` field, freshly generated on each GET of `/auth/login`. Add a *Regular Expression Extractor* as a child of the login-page sampler (Sampler → Add → Post-Processor → Regular Expression Extractor). Fill in:

Name of variable: `loginToken` Regex: `name="_token" value="([^\"]+)"` Template: `$1$`

Then in the login POST body, replace the recorded token value with `${loginToken}`.

- **Add HTTP Cookie Manager** under Test Plan. Each thread now gets its own cookie jar ⇒ 10 independent sessions instead of one shared mess. Test Plan → Add → Config Element → HTTP Cookie Manager.

Walkthrough — step 2: clean up + add assertions

Three fixes turn the raw recording into a real test:

- **Extract the CSRF token.** OrangeHRM's login POST includes a `_token` field, freshly generated on each GET of `/auth/login`. Add a *Regular Expression Extractor* as a child of the login-page sampler (Sampler → Add → Post-Processor → Regular Expression Extractor). Fill in:

Name of variable: `loginToken` Regex: `name="_token" value="([^\"]+)"` Template: `$1$`

Then in the login POST body, replace the recorded token value with `${loginToken}`.

- **Add HTTP Cookie Manager** under Test Plan. Each thread now gets its own cookie jar ⇒ 10 independent sessions instead of one shared mess. Test Plan → Add → Config Element → HTTP Cookie Manager.
- **Attach assertions** to “Submit credentials”: Response Assertion (body *contains* “Dashboard”) + Duration Assertion (< 3000 ms).

Walkthrough — step 2: clean up + add assertions

Three fixes turn the raw recording into a real test:

- **Extract the CSRF token.** OrangeHRM's login POST includes a `_token` field, freshly generated on each GET of `/auth/login`. Add a *Regular Expression Extractor* as a child of the login-page sampler (Sampler → Add → Post-Processor → Regular Expression Extractor). Fill in:

Name of variable: `loginToken` Regex: `name="_token" value="([^\"]+)"` Template: `$1$`

Then in the login POST body, replace the recorded token value with `${loginToken}`.

- **Add HTTP Cookie Manager** under Test Plan. Each thread now gets its own cookie jar ⇒ 10 independent sessions instead of one shared mess. Test Plan → Add → Config Element → HTTP Cookie Manager.
- **Attach assertions** to “Submit credentials”: Response Assertion (body *contains* “Dashboard”) + Duration Assertion (< 3000 ms).
- **Rename samplers** to human-readable names: “Open login page”, “Submit credentials”, “View dashboard”, “Open Admin module”, “Open PIM module”, “Logout”.

Walkthrough — step 3: the final test plan

Save it as `orangehrm.jmx`.

Walkthrough — step 4: run it from the CLI

From the terminal, in the same folder as `orangehrm.jmx`:

```
jmeter -n -t orangehrm.jmx -l orangehrm.jtl -e -o orangehrm_report/
```

What happens:

- 10 threads spin up over 10 seconds (one per second)
- Each replays the 6-request journey 5 times
- Total: **300 HTTP requests**
- Every sample lands in `orangehrm.jtl`
- When the run ends, JMeter builds `orangehrm_report/index.html`

Open the HTML report to see:

- APDEX (user satisfaction ratio)
- Per-request stats: mean, median, 90th/95th/99th percentiles, error %
- Response time *over time* chart
- Throughput chart
- Errors chart — ideally empty. If not, that's what the assertions caught.

**Best practices: before, during,
after**

Before the test

- **Get written permission from every stakeholder.** Load testing looks a lot like an attack. Get it in writing that you are allowed to do it. Include the ops team.

Before the test

- **Get written permission from every stakeholder.** Load testing looks a lot like an attack. Get it in writing that you are allowed to do it. Include the ops team.
- **Understand the application.** You cannot performance-test what you cannot describe. Which endpoints matter? Which are hot? Which are known slow?

Before the test

- **Get written permission from every stakeholder.** Load testing looks a lot like an attack. Get it in writing that you are allowed to do it. Include the ops team.
- **Understand the application.** You cannot performance-test what you cannot describe. Which endpoints matter? Which are hot? Which are known slow?
- **Write down the scenarios and success criteria.** What is being simulated? What load, over what time? What counts as “passed”?

Before the test

- **Get written permission from every stakeholder.** Load testing looks a lot like an attack. Get it in writing that you are allowed to do it. Include the ops team.
- **Understand the application.** You cannot performance-test what you cannot describe. Which endpoints matter? Which are hot? Which are known slow?
- **Write down the scenarios and success criteria.** What is being simulated? What load, over what time? What counts as “passed”?
- **Check the client machine.** 500 threads on a laptop measures the laptop, not the server.

Before the test

- **Get written permission from every stakeholder.** Load testing looks a lot like an attack. Get it in writing that you are allowed to do it. Include the ops team.
- **Understand the application.** You cannot performance-test what you cannot describe. Which endpoints matter? Which are hot? Which are known slow?
- **Write down the scenarios and success criteria.** What is being simulated? What load, over what time? What counts as “passed”?
- **Check the client machine.** 500 threads on a laptop measures the laptop, not the server.
- **Use a test environment that matches production.** Same OS, same JVM, same DB size, same network topology. If it can't match production, at least document how it differs.

Before the test

- **Get written permission from every stakeholder.** Load testing looks a lot like an attack. Get it in writing that you are allowed to do it. Include the ops team.
- **Understand the application.** You cannot performance-test what you cannot describe. Which endpoints matter? Which are hot? Which are known slow?
- **Write down the scenarios and success criteria.** What is being simulated? What load, over what time? What counts as “passed”?
- **Check the client machine.** 500 threads on a laptop measures the laptop, not the server.
- **Use a test environment that matches production.** Same OS, same JVM, same DB size, same network topology. If it can't match production, at least document how it differs.
- **Plan your test data** — and plan how to *clean it up* afterward.

Before the test

- **Get written permission from every stakeholder.** Load testing looks a lot like an attack. Get it in writing that you are allowed to do it. Include the ops team.
- **Understand the application.** You cannot performance-test what you cannot describe. Which endpoints matter? Which are hot? Which are known slow?
- **Write down the scenarios and success criteria.** What is being simulated? What load, over what time? What counts as “passed”?
- **Check the client machine.** 500 threads on a laptop measures the laptop, not the server.
- **Use a test environment that matches production.** Same OS, same JVM, same DB size, same network topology. If it can't match production, at least document how it differs.
- **Plan your test data** — and plan how to *clean it up* afterward.
- **Establish a baseline.** A load test result with nothing to compare against is a number without a unit.

Before the test

- **Get written permission from every stakeholder.** Load testing looks a lot like an attack. Get it in writing that you are allowed to do it. Include the ops team.
- **Understand the application.** You cannot performance-test what you cannot describe. Which endpoints matter? Which are hot? Which are known slow?
- **Write down the scenarios and success criteria.** What is being simulated? What load, over what time? What counts as “passed”?
- **Check the client machine.** 500 threads on a laptop measures the laptop, not the server.
- **Use a test environment that matches production.** Same OS, same JVM, same DB size, same network topology. If it can't match production, at least document how it differs.
- **Plan your test data** — and plan how to *clean it up* afterward.
- **Establish a baseline.** A load test result with nothing to compare against is a number without a unit.
- **Agree on a report format** in advance. The report exists to be read; someone should agree ahead of time what it should say.

During the test

- **Don't share the test environment.** Someone deploying to it during your run contaminates the whole result set. Put a sign on the door.

During the test

- **Don't share the test environment.** Someone deploying to it during your run contaminates the whole result set. Put a sign on the door.
- **Make the test realistic.** Real users pause between clicks. Add **think time** with Timers. A test with zero think time is a stress test wearing a load test's hat.

During the test

- **Don't share the test environment.** Someone deploying to it during your run contaminates the whole result set. Put a sign on the door.
- **Make the test realistic.** Real users pause between clicks. Add **think time** with Timers. A test with zero think time is a stress test wearing a load test's hat.
- **Give the ramp-up some room.** Never start every user at $t = 0$.

During the test

- **Don't share the test environment.** Someone deploying to it during your run contaminates the whole result set. Put a sign on the door.
- **Make the test realistic.** Real users pause between clicks. Add **think time** with Timers. A test with zero think time is a stress test wearing a load test's hat.
- **Give the ramp-up some room.** Never start every user at $t = 0$.
- **Monitor the client.** If JMeter itself runs out of CPU or memory, the numbers it produces are about JMeter, not about the server.

During the test

- **Don't share the test environment.** Someone deploying to it during your run contaminates the whole result set. Put a sign on the door.
- **Make the test realistic.** Real users pause between clicks. Add **think time** with Timers. A test with zero think time is a stress test wearing a load test's hat.
- **Give the ramp-up some room.** Never start every user at $t = 0$.
- **Monitor the client.** If JMeter itself runs out of CPU or memory, the numbers it produces are about JMeter, not about the server.
- **Monitor the server too.** CPU, memory, network, disk I/O, database wait times, GC pauses. The interesting failures are almost never in the HTTP status code.

During the test

- **Don't share the test environment.** Someone deploying to it during your run contaminates the whole result set. Put a sign on the door.
- **Make the test realistic.** Real users pause between clicks. Add **think time** with Timers. A test with zero think time is a stress test wearing a load test's hat.
- **Give the ramp-up some room.** Never start every user at $t = 0$.
- **Monitor the client.** If JMeter itself runs out of CPU or memory, the numbers it produces are about JMeter, not about the server.
- **Monitor the server too.** CPU, memory, network, disk I/O, database wait times, GC pauses. The interesting failures are almost never in the HTTP status code.
- **Watch for anomalies live.** If error rate spikes, don't wait for the report — check the server logs immediately.

During the test

- **Don't share the test environment.** Someone deploying to it during your run contaminates the whole result set. Put a sign on the door.
- **Make the test realistic.** Real users pause between clicks. Add **think time** with Timers. A test with zero think time is a stress test wearing a load test's hat.
- **Give the ramp-up some room.** Never start every user at $t = 0$.
- **Monitor the client.** If JMeter itself runs out of CPU or memory, the numbers it produces are about JMeter, not about the server.
- **Monitor the server too.** CPU, memory, network, disk I/O, database wait times, GC pauses. The interesting failures are almost never in the HTTP status code.
- **Watch for anomalies live.** If error rate spikes, don't wait for the report — check the server logs immediately.
- **Communicate.** If something is on fire, tell somebody now, not in the report.

During the test

- **Don't share the test environment.** Someone deploying to it during your run contaminates the whole result set. Put a sign on the door.
- **Make the test realistic.** Real users pause between clicks. Add **think time** with Timers. A test with zero think time is a stress test wearing a load test's hat.
- **Give the ramp-up some room.** Never start every user at $t = 0$.
- **Monitor the client.** If JMeter itself runs out of CPU or memory, the numbers it produces are about JMeter, not about the server.
- **Monitor the server too.** CPU, memory, network, disk I/O, database wait times, GC pauses. The interesting failures are almost never in the HTTP status code.
- **Watch for anomalies live.** If error rate spikes, don't wait for the report — check the server logs immediately.
- **Communicate.** If something is on fire, tell somebody now, not in the report.



If your test environment doubles as production, your tests will eventually double as outages.

After the test

- **Analyse first, react second.** A single ugly number is not a conclusion. Look at percentiles, not just averages; look at trends across the run, not just totals.

After the test

- **Analyse first, react second.** A single ugly number is not a conclusion. Look at percentiles, not just averages; look at trends across the run, not just totals.
- **Document the results** in the agreed format — environment details, test plan version, load profile, baselines used for comparison.

After the test

- **Analyse first, react second.** A single ugly number is not a conclusion. Look at percentiles, not just averages; look at trends across the run, not just totals.
- **Document the results** in the agreed format — environment details, test plan version, load profile, baselines used for comparison.
- **Share the report with the team;** Devs need the detail; ops needs the timings; product needs the summary.

After the test

- **Analyse first, react second.** A single ugly number is not a conclusion. Look at percentiles, not just averages; look at trends across the run, not just totals.
- **Document the results** in the agreed format — environment details, test plan version, load profile, baselines used for comparison.
- **Share the report with the team;** Devs need the detail; ops needs the timings; product needs the summary.
- **Clean up test data.** Nothing embarrasses like a production DB full of `loadtest_user_00042`.

After the test

- **Analyse first, react second.** A single ugly number is not a conclusion. Look at percentiles, not just averages; look at trends across the run, not just totals.
- **Document the results** in the agreed format — environment details, test plan version, load profile, baselines used for comparison.
- **Share the report with the team;** Devs need the detail; ops needs the timings; product needs the summary.
- **Clean up test data.** Nothing embarrasses like a production DB full of `loadtest_user_00042`.
- **Write down what you learned**, for yourself and the next person. Include what *didn't* work: which endpoints misled, which listeners were useless, which flags you always forgot.

After the test

- **Analyse first, react second.** A single ugly number is not a conclusion. Look at percentiles, not just averages; look at trends across the run, not just totals.
- **Document the results** in the agreed format — environment details, test plan version, load profile, baselines used for comparison.
- **Share the report with the team;** Devs need the detail; ops needs the timings; product needs the summary.
- **Clean up test data.** Nothing embarrasses like a production DB full of `loadtest_user_00042`.
- **Write down what you learned**, for yourself and the next person. Include what *didn't* work: which endpoints misled, which listeners were useless, which flags you always forgot.

❓ Why do we care?

Performance testing is a repeated activity, not a one-off event. The value compounds when future-you can compare against past-you's baselines. Save the raw `.jtl` files — they are small and priceless.

Thank You!